

GENRE PACK · 5 LESSONS

TYCOON

Build a Tycoon Players Can't Stop Playing

YOU'LL SHIP

A multiplayer tycoon that saves progress, with rebirths and two game passes.

- T.1 The foundation: plots and player data
- T.2 Purchase buttons
- T.3 Droppers, conveyors and the collector
- T.4 Rebirth: the long-term loop
- T.5 Game passes, polish and launch

Tycoons are one of Roblox's biggest genres for a simple reason: **watching your empire grow feels amazing**. Every few seconds something gets better. In this pack you'll build a complete, saving tycoon with plots, purchase buttons, droppers, conveyors, rebirths and fair game passes: *Pizza Tycoon*.

How to use this pack

- **Before you start:** finish the Core Course up to Module 6 (or know ModuleScripts, attributes and DataStores).
- **Build in a new place** (File → New → Baseplate).
- The scripts are split by system: **TycoonData** (cash and saving), **Plots** (claiming), **Buttons** (buying), **Droppers** (money-making) and **Rebirth**. Each one is short and does one job.

Every script says where it lives in its first line, for example `-- ServerScriptService/Plots`.

T.1 The foundation: plots and player data

Build one plot

Everything a player owns lives inside their plot. Build one, then copy it.

1. In Workspace, create a Folder `Plots` and a **Model** `Plot1` inside it.
2. Inside `Plot1` add:
 - `Base` : a large flat anchored Part (e.g. `80, 1, 80`).
 - `ClaimPad` : a glowing anchored Part at the entrance, with a BillboardGui showing "CLAIM".
 - `CollectPad` : an anchored Part where the owner collects cash.
 - `Collector` : an anchored Part at the end of the conveyor (built in T.3).
 - Two empty Folders: `Buttons` and `Items` .
3. When your plot is finished (after T.3), duplicate it into `Plot2` ... `Plot6` around a central area. Six plots is a good server size.

Player data that saves

Tycoons live on saved progress: if a player loses their pizza empire once, they never come back. So the data module comes first. It keeps each player's cash, bought items and rebirths, mirrors cash in the leaderboard, and saves safely using the same patterns as Core Course Module 6.

```
-- ServerScriptService/TycoonData (ModuleScript)
local DataStoreService = game:GetService("DataStoreService")
local MarketplaceService = game:GetService("MarketplaceService")
local Players = game:GetService("Players")

local store = DataStoreService:GetDataStore("PizzaTycoon_v1")
```

```

-- Game Pass IDs, filled in during T.5 (0 = not set up yet)
local PASSES = {
    DoubleCash = 0,
    AutoCollect = 0,
}

local TycoonData = {}
local profiles = {} -- [player] = { Cash, Owned, Rebirths }
local passCache = {} -- [player] = { [passName] = true }

local function key(player: Player): string
    return "Player_" .. player.UserId
end

local function updateStats(player: Player)
    local profile = profiles[player]
    local stats = player:FindFirstChild("leaderstats")
    if profile and stats then
        stats.Cash.Value = profile.Cash
        stats.Rebirths.Value = profile.Rebirths
    end
end

local function load(player: Player)
    local data
    for attempt = 1, 3 do
        local ok, result = pcall(store.GetAsync, store, key(player))
        if ok then
            data = result or {}
            break
        end
        warn("Load failed:", result)
        task.wait(2 * attempt)
    end
    if data == nil then
        player:Kick("We couldn't load your tycoon. Please rejoin in a minute.")
        return
    end

    local stats = Instance.new("Folder")
    stats.Name = "leaderstats"
    for _, name in { "Cash", "Rebirths" } do
        local value = Instance.new("IntValue")
        value.Name = name
    end
end

```

```

        value.Parent = stats
    end
    stats.Parent = player

    passCache[player] = {}
    for name, id in PASSES do
        if id ~= 0 then
            local ok, owns = pcall(MarketplaceService.UserOwnsGamePassAsync,
MarketplaceService, player.UserId, id)
            passCache[player][name] = ok and owns
        end
    end

    profiles[player] = {
        Cash = data.Cash or 0,
        Owned = data.Owned or {},
        Rebirths = data.Rebirths or 0,
    }
    updateStats(player)
end

local function save(player: Player)
    local profile = profiles[player]
    if not profile then return end
    local ok, err = pcall(store.UpdateAsync, store, key(player), function()
        return { Cash = profile.Cash, Owned = profile.Owned, Rebirths = profile.Rebirths }
    end)
    if not ok then
        warn("Save failed for", player.Name, err)
    end
end

-- Public API -----

-- Waits until the player's data is loaded. Returns nil if they left.
function TycoonData.waitFor(player: Player)
    while player.Parent and not profiles[player] do
        task.wait(0.1)
    end
    return profiles[player]
end

function TycoonData.addCash(player: Player, amount: number)
    local profile = profiles[player]

```

```

    if not profile then return end
    profile.Cash += amount
    updateStats(player)
end

function TycoonData.spend(player: Player, amount: number): boolean
    local profile = profiles[player]
    if not profile or profile.Cash < amount then return false end
    profile.Cash -= amount
    updateStats(player)
    return true
end

function TycoonData.markOwned(player: Player, itemName: string)
    local profile = profiles[player]
    if profile and not table.find(profile.Owned, itemName) then
        table.insert(profile.Owned, itemName)
    end
end

function TycoonData.getOwned(player: Player): { string }
    local profile = profiles[player]
    return profile and profile.Owned or {}
end

function TycoonData.hasPass(player: Player, name: string): boolean
    return passCache[player] ~= nil and passCache[player][name] == true
end

function TycoonData.getMultiplier(player: Player): number
    local profile = profiles[player]
    if not profile then return 1 end
    local multiplier = 1 + 0.5 * profile.Rebirths -- every rebirth: +50% forever
    if TycoonData.hasPass(player, "DoubleCash") then
        multiplier *= 2
    end
    return multiplier
end

-- Wipes cash and items for a permanent multiplier. Returns true on success.
function TycoonData.rebirth(player: Player, cost: number): boolean
    local profile = profiles[player]
    if not profile or profile.Cash < cost then return false end
    profile.Cash = 0

```

```

    profile.Owned = {}
    profile.Rebirths += 1
    updateStats(player)
    save(player)
    return true
end

-- Lifecycle -----

MarketplaceService.PromptGamePassPurchaseFinished:Connect(function(player, passId,
purchased)
    if not purchased or not passCache[player] then return end
    for name, id in PASSES do
        if id == passId then
            passCache[player][name] = true
        end
    end
end)

Players.PlayerAdded:Connect(load)
for _, player in Players:GetPlayers() do
    task.spawn(load, player)
end

Players.PlayerRemoving:Connect(function(player)
    save(player)
    profiles[player] = nil
    passCache[player] = nil
end)

game:BindToClose(function()
    local pending = 0
    for _, player in Players:GetPlayers() do
        pending += 1
        task.spawn(function()
            save(player)
            pending -= 1
        end)
    end
    while pending > 0 do
        task.wait()
    end
end)

```

```

task.spawn(function()
    while true do
        task.wait(60)
        for _, player in Players:GetPlayers() do
            task.spawn(save, player)
        end
    end
end)

return TycoonData

```

WATCH OUT To test saving in Studio, publish the place and turn on **Game Settings** → **Security** → **Enable Studio Access to API Services**. Without it, every load fails and the player gets kicked (that's the safe behavior).

Claiming a plot

```

-- ServerScriptService/Plots (Script)
local Players = game:GetService("Players")
local ServerScriptService = game:GetService("ServerScriptService")

local TycoonData = require(ServerScriptService.TycoonData)
local Buttons = require(ServerScriptService.Buttons)

local plotOf = {} -- [player] = plot

local function setClaimPad(plot: Model, visible: boolean)
    plot.ClaimPad.Transparency = if visible then 0 else 1
    plot.ClaimPad.CanTouch = visible
    local gui = plot.ClaimPad:FindFirstChildOfClass("BillboardGui")
    if gui then gui.Enabled = visible end
end

for _, plot in workspace.Plots:GetChildren() do
    plot:SetAttribute("Owner", 0)
    Buttons.setup(plot)

    plot.ClaimPad.Touched:Connect(function(hit)
        local player = Players:GetPlayerFromCharacter(hit.Parent)
        if not player or plotOf[player] or plot:GetAttribute("Owner") ~= 0 then return end
        plotOf[player] = plot
        plot:SetAttribute("Owner", player.UserId)
        setClaimPad(plot, false)
    end)
end

```

```

    if TycoonData.waitFor(player) then
        Buttons.restore(plot, player) -- rebuild everything they bought before
    end
end)
end

Players.PlayerRemoving:Connect(function(player)
    local plot = plotOf[player]
    if not plot then return end
    plotOf[player] = nil
    plot:SetAttribute("Owner", 0)
    Buttons.clear(plot)
    setClaimPad(plot, true)
end)

```

The plot's `Owner` attribute (a `UserId`, `0` = free) is the single source of truth. Every other system just reads it.

TRY IT

1. Create the two ModuleScripts `TycoonData` and `Buttons` (`Buttons` stays empty until T.2: for now put `return { setup = function() end, restore = function() end, clear = function() end }` in it).
2. Playtest: touch a claim pad. It should disappear, and the leaderboard should show Cash and Rebirths.
3. Leave and rejoin (published place with API access on): no errors in Output.

T.2 Purchase buttons

The core loop of every tycoon: **walk onto a button, pay, something new appears**. Players should afford their first button within 10 seconds and see a new button right after.

How it's built

- Every purchasable thing (a counter, an oven, a dropper, a wall) is a **Model inside the plot's `Items` folder**, built right where it should appear.
- Give each item two attributes: `Price` (number) and optionally `Requires` (string: the name of the item that must be bought first).
- For each item, put a button Part in the `Buttons` folder **with exactly the same name**. Add a BillboardGui with a TextLabel named `Label` to show name and price.

At startup, the script moves all items into storage and brings them back when bought.

```

-- ServerScriptService/Buttons (ModuleScript)
local Players = game:GetService("Players")
local ServerStorage = game:GetService("ServerStorage")

local TycoonData = require(script.Parent.TycoonData)

local Buttons = {}
local storageOf = {} -- [plot] = Folder with the items not bought yet

local function showButton(button: BasePart, visible: boolean)
    button.Transparency = if visible then 0 else 1
    button.CanCollide = false
    button.CanTouch = visible
    local gui = button:FindFirstChildOfClass("BillboardGui")
    if gui then gui.Enabled = visible end
end

-- Shows the buttons whose item isn't bought yet and whose requirement is met
local function refresh(plot: Model)
    local storage = storageOf[plot]
    local hasOwner = plot:GetAttribute("Owner") ~= 0
    for _, button in plot.Buttons:GetChildren() do
        local item = storage:FindFirstChild(button.Name)
        local requires = item and item:GetAttribute("Requires")
        local unlocked = item ~= nil and (requires == nil or
plot.Items:FindFirstChild(requires) ~= nil)
        showButton(button, hasOwner and unlocked)
    end
end

local function place(plot: Model, itemName: string)
    local item = storageOf[plot]:FindFirstChild(itemName)
    if item then
        item.Parent = plot.Items
    end
end

function Buttons.setup(plot: Model)
    local storage = Instance.new("Folder")
    storage.Name = plot.Name .. "_Storage"
    storage.Parent = ServerStorage
    storageOf[plot] = storage
end

```

```

for _, item in plot.Items:GetChildren() do
    item.Parent = storage
end

for _, button in plot.Buttons:GetChildren() do
    local item = storage:FindFirstChild(button.Name)
    if not item then
        warn("Button has no matching item:", button:GetFullName())
        continue
    end
    local price = item:GetAttribute("Price") or 0
    local label = button:FindFirstChild("Label", true)
    if label then
        label.Text = ("%s\n$%d"):format(button.Name, price)
    end

    local busy = false
    button.Touched:Connect(function(hit)
        if busy then return end
        local player = Players:GetPlayerFromCharacter(hit.Parent)
        if not player or player.UserId ~= plot:GetAttribute("Owner") then return end
        if not storage:FindFirstChild(button.Name) then return end -- already bought
        busy = true
        if TycoonData.spend(player, price) then
            TycoonData.markOwned(player, button.Name)
            place(plot, button.Name)
            refresh(plot)
        end
        task.wait(0.5)
        busy = false
    end)
end
refresh(plot)
end

-- Rebuilds a returning player's tycoon
function Buttons.restore(plot: Model, player: Player)
    for _, itemName in TycoonData.getOwned(player) do
        place(plot, itemName)
    end
    refresh(plot)
end

-- Puts every item back into storage (player left, or rebirth)

```

```
function Buttons.clear(plot: Model)
  for _, item in plot.Items:GetChildren() do
    item.Parent = storage0f[plot]
  end
  refresh(plot)
end

return Buttons
```

Why this design works:

- Only the owner can press their buttons (`player.UserId ~= Owner` check), and the **server** spends the cash.
- `Requires` builds the progression tree: *Counter* → *Oven 1* → *Conveyor* → *Oven 2*... without any extra code.
- Restoring saved progress is just "place every owned item again".

Pricing that keeps players hooked

Use a growth factor of about **×1.6** between buttons and make the first purchase cost almost nothing:

#	Item	Price	Requires
1	Counter	0	none
2	Oven 1 (dropper)	25	Counter
3	Conveyor	40	Oven 1
4	Oven 2	65	Conveyor
5	Walls	100	Counter
6	Oven 3	170	Oven 2
...	keep ×1.6	...	...

TIP Always show 2–3 buttons at once. A choice ("oven or walls?") feels better than a single path, and players plan their next purchase while waiting.

TRY IT

1. Build a Counter, an Oven and some decorations as items with `Price` and `Requires`.
2. Add a button for each item, with matching names.
3. Playtest: claim, buy the free Counter, and check the Oven button appears. (You'll earn cash for it in the next lesson. For now, test by temporarily giving yourself cash:

```
TycoonData.addCash(player, 1000) after waitFor .)
```

T.3 Droppers, conveyors and the collector

This is the money machine: **droppers** spawn pizzas, **conveyors** carry them, the **collector** turns them into cash, and the owner picks the cash up at the **collect pad**.

Set it up:

1. A dropper is an item with two attributes: `DropEvery` (seconds, e.g. `2`) and `DropValue` (cash per drop, e.g. `5`). Inside it, add a Part named `Spout` where drops come out.
2. A conveyor is any anchored Part tagged `Conveyor` (Tag Editor, or the **Tags** section in Properties). Optional attribute `Speed` (default 8). It moves things along its **front face**, so rotate it to point at the collector.
3. Add a SurfaceGui with a TextLabel named `Label` on the `CollectPad` to show waiting cash.

```
-- ServerScriptService/Droppers (Script)
local CollectionService = game:GetService("CollectionService")
local Debris = game:GetService("Debris")
local Players = game:GetService("Players")
local ServerScriptService = game:GetService("ServerScriptService")

local TycoonData = require(ServerScriptService.TycoonData)

local MAX_DROPS_PER_PLOT = 60 -- protects the server if nobody collects

-- Conveyors: an anchored part with a velocity pushes whatever touches it
for _, conveyor in CollectionService:GetTagged("Conveyor") do
    conveyor.AssemblyLinearVelocity = conveyor.CFrame.LookVector *
(conveyor:GetAttribute("Speed") or 8)
end

local function ownerOf(plot: Model): Player?
    return Players:GetPlayerByUserId(plot:GetAttribute("Owner"))
end

local function drop(plot: Model, dropper: Model)
```

```

local spout = dropper:FindFirstChild("Spout", true)
if not spout or #plot.Drops:GetChildren() >= MAX_DROPS_PER_PLOT then return end
local part = Instance.new("Part")
part.Size = Vector3.new(1.2, 0.4, 1.2)
part.Color = Color3.fromRGB(255, 200, 90)
part.Material = Enum.Material.SmoothPlastic
part.CFrame = spout.CFrame * CFrame.new(0, -1, 0)
part:SetAttribute("DropValue", dropper:GetAttribute("DropValue") or 1)
part.Parent = plot.Drops
part:SetNetworkOwner(nil)
Debris:AddItem(part, 20) -- anything that falls off the conveyor cleans itself up
end

for _, plot in workspace.Plots:GetChildren() do
    local drops = Instance.new("Folder")
    drops.Name = "Drops"
    drops.Parent = plot
    plot:SetAttribute("Stored", 0)

    local label = plot.CollectPad:FindFirstChild("Label", true)
    plot:GetAttributeChangedSignal("Stored"):Connect(function()
        if label then label.Text = ("%d"):format(plot:GetAttribute("Stored")) end
    end)

    -- Owner left: clear the conveyor and the waiting cash
    plot:GetAttributeChangedSignal("Owner"):Connect(function()
        if plot:GetAttribute("Owner") == 0 then
            drops:ClearAllChildren()
            plot:SetAttribute("Stored", 0)
        end
    end)

    plot.Collector.Touched:Connect(function(hit)
        local value = hit:GetAttribute("DropValue")
        if not value or not hit:IsDescendantOf(drops) then return end
        hit:Destroy()
        local owner = ownerOf(plot)
        if not owner then return end
        local amount = math.floor(value * TycoonData.getMultiplier(owner))
        if TycoonData.hasPass(owner, "AutoCollect") then
            TycoonData.addCash(owner, amount) -- goes straight to their wallet
        else
            plot:SetAttribute("Stored", plot:GetAttribute("Stored") + amount)
        end
    end)
end

```

```

end)

plot.CollectPad.Touched:Connect(function(hit)
    local owner = ownerOf(plot)
    if not owner or Players:GetPlayerFromCharacter(hit.Parent) ~= owner then return
end
    local stored = plot:GetAttribute("Stored")
    if stored > 0 then
        plot:SetAttribute("Stored", 0)
        TycoonData.addCash(owner, stored)
    end
end)
end

-- One loop runs every dropper on every claimed plot
local lastDrop = {}
while true do
    local now = os.clock()
    for _, plot in workspace.Plots:GetChildren() do
        if plot:GetAttribute("Owner") ~= 0 then
            for _, item in plot.Items:GetChildren() do
                local every = item:GetAttribute("DropEvery")
                if every and now - (lastDrop[item] or 0) >= every then
                    lastDrop[item] = now
                    drop(plot, item)
                end
            end
        end
    end
    task.wait(0.2)
end
end

```

Performance rules for tycoons (servers with 6 busy plots create a lot of parts):

- `MAX_DROPS_PER_PLOT` and `Debris` keep the part count under control.
- Drops are simple Parts, not meshes with scripts inside.
- `SetNetworkOwner(nil)` keeps physics on the server, so drops don't stutter between players.

TIP Make drops look like the product: a flat golden disc reads as "pizza" from far away. Add a satisfying sound on the collector and a coin sound on the collect pad.

TRY IT

1. Set the Oven's `DropEvery` to 2 and `DropValue` to 5, place its `Spout` over a tagged conveyor that leads to the collector.
2. Playtest: pizzas should drop, ride the conveyor and increase the number on the collect pad.
3. Step on the collect pad: cash moves to the leaderboard. Buy the next button with it.



Rebirth: the long-term loop

After 20–40 minutes a player owns everything. Without a reason to continue, they leave. **Rebirth** gives them one: reset your tycoon, keep a permanent bonus (+50% cash per rebirth), and rebuild faster than before.

Set it up: in `ReplicatedStorage`, create a Folder `Remotes` with a RemoteEvent `Rebirth`.

```
-- ServerScriptService/Rebirth (Script)
local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local ServerScriptService = game:GetService("ServerScriptService")

local TycoonData = require(ServerScriptService.TycoonData)
local Buttons = require(ServerScriptService.Buttons)

local remote = ReplicatedStorage:WaitForChild("Remotes"):WaitForChild("Rebirth")
local lastRequest = {}

local function rebirthCost(rebirths: number): number
    return 10000 * (rebirths + 1) -- 10k, 20k, 30k, ...
end

local function plotOf(player: Player): Model?
    for _, plot in workspace.Plots:GetChildren() do
        if plot:GetAttribute("Owner") == player.UserId then
            return plot
        end
    end
    return nil
end

remote.OnServerEvent:Connect(function(player)
    -- rate limit: one request per 2 seconds
    if lastRequest[player] and os.clock() - lastRequest[player] < 2 then return end
    lastRequest[player] = os.clock()
```

```

    local plot = plotOf(player)
    local stats = player:FindFirstChild("leaderstats")
    if not plot or not stats then return end

    if TycoonData.rebirth(player, rebirthCost(stats.Rebirths.Value)) then
        plot:SetAttribute("Stored", 0)
        plot.Drops:ClearAllChildren()
        Buttons.clear(plot) -- items go back to storage, the first buttons reappear
    end
end)

Players.PlayerRemoving:Connect(function(player)
    lastRequest[player] = nil
end)

```

The client only *asks*. The server checks the price from its own data, so an exploiter can't rebirth for free.

The button. Add a TextButton `RebirthButton` to your HUD with this LocalScript inside:

```

-- StarterGui/HUD/RebirthButton/RebirthClient (LocalScript)
local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")

local button = script.Parent
local remote = ReplicatedStorage:WaitForChild("Remotes"):WaitForChild("Rebirth")
local stats = Players.LocalPlayer:WaitForChild("leaderstats")

local function update()
    local cost = 10000 * (stats.Rebirths.Value + 1)
    button.Text = ("REBIRTH ($%d) · +50%% forever"):format(cost)
    button.AutoButtonColor = stats.Cash.Value >= cost
    button.BackgroundTransparency = if stats.Cash.Value >= cost then 0 else 0.5
end

stats.Cash.Changed:Connect(update)
stats.Rebirths.Changed:Connect(update)
update()

button.Activated:Connect(function()
    remote:FireServer()
end)

```

TIP Pick the rebirth cost so the first rebirth takes 20–30 minutes. The second run should be noticeably faster: that "I'm so much faster now" feeling is what brings players back tomorrow.

TRY IT

1. Temporarily set the cost to 100 to test quickly.
2. Rebirth: cash resets, items disappear, the first buttons return and Rebirths shows 1.
3. Check pizzas now give 50% more cash. Set the cost back afterwards.

T.5 Game passes, polish and launch

Two passes players actually want

The data module already supports both. You only create them and paste the IDs.

1. create.roblox.com → **Creations** → **your experience** → **Monetization** → **Passes**. Create **2× Cash** and **Auto Collect**, set them for sale, copy their IDs.
2. Paste the IDs into `PASSES` at the top of `TycoonData`.
3. Add a shop button to the HUD:

```
-- StarterGui/HUD/ShopButtons (LocalScript, next to two TextButtons)
local MarketplaceService = game:GetService("MarketplaceService")
local Players = game:GetService("Players")

local DOUBLE_CASH = 0 -- same IDs as in TycoonData
local AUTO_COLLECT = 0

script.Parent.DoubleCashButton.Activated:Connect(function()
    MarketplaceService.PromptGamePassPurchase(Players.LocalPlayer, DOUBLE_CASH)
end)

script.Parent.AutoCollectButton.Activated:Connect(function()
    MarketplaceService.PromptGamePassPurchase(Players.LocalPlayer, AUTO_COLLECT)
end)
```

`TycoonData` listens to `PromptGamePassPurchaseFinished`, so a pass works the moment it's bought, without rejoining.

Pass	Suggested price	Why it's fair
2× Cash	199–299 R\$	Faster, but free players reach the same places
Auto Collect	99–149 R\$	Convenience, not power
VIP area with chat tag	149 R\$	Social status

WATCH OUT

Never sell "skip the whole tycoon". Players who skip the build have nothing left to do and quit, and the free players feel the game is pay-to-win.

Polish checklist

- A sound for every purchase (a "ka-ching" or a build sound) and a small particle burst where the item appears.
- Make new items **pop in**: tween them from slightly below the floor, or from scale 0.8 to 1.
- An arrow or glowing path to the next button for brand-new players.
- A floating "+\$25" text above the collect pad when cash is picked up.

Launch tips for tycoons

- **Thumbnail**: a finished, colorful plot from above, with lots of stuff. Tycoon players buy the fantasy of the full build.
- **First session**: first purchase within 10 s, a dropper within 60 s, and one visible "big" item (a delivery truck, a giant pizza sign) within 5 minutes.
- **Updates**: add a new floor or product line every 1–2 weeks. Tycoon players come back for "new stuff to buy".

TRY IT

1. Create the passes (you can test them in Studio: Studio purchases are free test purchases).
2. Buy 2× Cash in a playtest and check that pizzas give double cash immediately.
3. Buy Auto Collect and check that cash goes straight to the leaderboard.

Your Pizza Tycoon checklist

- ☐ 6 plots with claim pads, and claiming works
- ☐ Data saves on leave, on autosave and on shutdown
- ☐ Buttons appear in order through **Requires**
- ☐ Droppers, conveyors, collector and collect pad make money
- ☐ Rebirth resets the plot and adds +50% permanently
- ☐ 2× Cash and Auto Collect passes work without rejoining
- ☐ First purchase within 10 seconds for a new player

☐ Tested with 2 players (Test → Clients and Servers)